

Oasys Git Hotfixing Procedure

The Definitive Neil Edition v1.0.2

Note that the following method does not utilize any external utilities such as SmartGit, nor Git Flow, although the Git Flow naming conventions are adhered to.

I. Create hotfix branch off of master branch

- a) `git checkout master`
 - b) `git checkout -b hotfix/3.x.xx`
 - c) **optional** `git push --set-upstream origin hotfix/3.x.xx`
- i. If this hotfix requires a long integration period, or a heavy number of modifications, it is a good idea to have a remote upstream to origin as a backup in case something were to happen to your local repository copy. Note, however, that setting an upstream is NOT a mandatory step for a hotfix, and is completely up to the developer.

II. Perform the codefix updates in the hotfix branch

Presented herein are several different methods of performing the updates in the hotfix branch -- the optimal methods are usually one of the merge methods (a) or (b). Ideally, the "source branch" used in references below is a separate branch off of the develop which contains all the necessary updates by itself. It's not ideal to use the "develop" branch itself, as it may contain other mergable updates that we do not want for the targeted hotfix; this is however mitigated by careful and individual selection of which changes will be merged into the hotfix branch.

- a) **optional MERGETOOL METHOD**: My preferred method is utilizing a 3rd party merge tool which allows for visual inspection and selection of the changes to integrate (I use BeyondCompare). The mergetool must be configured separately at some point, and is beyond the scope of this document.
- i. Merging a single file: `git mergetool source_branch -- path_to_source_file`
 - ii. Merging multiple files: `git mergetool source_branch -- source_file_1 source_file_2 some_wildcard_expression etc1 etc2 ...`
 - iii. Merging against an entire branch: `git mergetool source_branch`

This method will look for any and all required merges between your hotfix branch and the source fix branch. This is generally a good approach to use to ensure you do not accidentally exclude any files required for your hotfix.

- b) **optional TRADITIONAL MERGE METHOD**: A balance between ease and functionality, this method will try and automatically (*automagically*) resolve the differentials needed to backport and update the hotfix branch. Works 99% of the time flawlessly. The syntax and format is identical to "mergetool" above, except for using the "merge" sub-command instead of "mergetool".
- i. If a conflict is found during a merge, you must:
 - 1. Open the problematic file
 - 2. Edit/fix conflicts manually
 - 3. Save and close file
 - 4. `git add path_to_conflicted_file`
 - 5. `git merge --continue`

- c) **optional** **NOT RECOMMENDED** **CHECKOUT METHOD:** If you do not have, or wish not to use a mergetool, or standard merge, you can still run a raw comparison and review each difference between 2 branches, and manually integrate the differences between your hotfix and develop fix branches. This method does not use the merge tool's automagic capabilities, and shows full differences between files, so beware! The format of the command is similar to the mergetool method in relation to selecting a single file, or doing a whole branch comparison:
- i. Checking out to patch single file with review:
git checkout --patch source_branch -- path_to_source_file
 - ii. Checking out to patch multiple files with review:
git checkout --patch source_branch -- file1 file2 file3 wildcard_expression ...
 - iii. Checking out to patch entire branch with review:
git checkout --patch source_branch
- Since the checkout command does not use the intelligent merge black magic logic, so it will go through every single difference, regardless of if it's a valid merge candidate or not!
- d) **optional** **MANUAL UPDATE:** If the fix is simple enough, it is quickest and easiest to simply manually update the hotfix branch with the required fix. Caution should of course be exercised to ensure the updated line(s) of code match the develop fix branch exactly to avoid potential future merge conflicts. With this method, it is still recommended to use **git diff** or **git difftool** to individually compare affected files, and bring in changes line by line.

!NOTE! DO NOT COMMIT ANY CHANGES UNTIL AFTER THE NEXT STEP!

III. Version number increment and hotfix commit

- a) Open **oasys_ver.txt** and increment the version value(s) as necessary
- b) Save the file, then close
- c) **git add oasys_ver.txt**

IV. Hotfix commit

- a) **git commit -am "Hotfix 3.x.xx fixes bug(s) OA-xxxx, OA-yyyy"**

* We do not have a standard message text template for hotfixes, but this general example, or something close to it, should be suitable.

V. Merge hotfix branch into master branch

- a) **git checkout master**
- b) **git merge hotfix/3.x.xx**
- c) If you have a conflict with **oasys_ver.txt**, refer to steps listed in section II.b)i

VI. Apply new tag to commit

- a) While still in master branch, type **git tag 3.x.xx** (the NEW version for this hotfix application)

VII. Updating develop branch with tag and changes

- a) **git checkout develop**
- b) **git merge master**
- c) Conflict should arise for **oasys_ver.txt** -- fix with new version value (and ensure to keep the "beta" string, and have the last 2 numbers of the hotfix after "beta" (e.g., **v=3.4 beta 26**), as this string is used in the Oasys code to make various determinations). Refer again to II.b)i if necessary for specific conflict resolution procedure.

VIII. Push everything to origin

- a) **git checkout master**
- b) **git push**
- c) **git push --tags**
- d) **git checkout develop**
- e) **git push**
- f) **git push --tags**

* The multiple "push all" and "push tags" routines are probably not 100% necessary, but it does not hurt to do so to ensure full upstream synchronization!

IX. Cleanup

- a) When new fixes have been validated in master and develop, you may remove the old hotfix branch.
 - i. You can be checked out to any branch except for the hotfix branch you want to remove
 - ii. **git branch -d hotfix/3.x.xx**
 - iii. **git push --delete origin hotfix/3.x.xx** (only needed if you performed an upstream push earlier when creating the hotfix branch).

The very TL;DR version

- 1) start hotfix branch from master
- 2) do the changes
- 3) change version number in oasys_ver.txt
- 4) commit the changes in the hotfix branch
- 5) merge hotfix -> master
- 6) apply new tag (lightweight with just version value) `git tag 3.3.x`
- 7) checkout develop
- 8) merge from master (this in theory should bring over the tag also)
- 9) fix oasys_ver.txt in develop
- 10) git push --all (master/develop)
- 11) git push --tags (master/develop)
- 12) cleanup old hotfix branch(es)